TASK - 13 MARIADB DATABASE BACKUP TO AMAZON S3

OBJECTIVE:

1. Create an IAM Role that allows the EC2 instance to access an Amazon S3 bucket.
2. Configure a MariaDB server on the EC2 instance.
3. Create a shell script to take backups of the MariaDB database into a backup directory.
4. Configure AWS CLI on the EC2 instance and synchronize the backup directory with an Amazon S3 bucket.
5. Verify that the MariaDB backup files are successfully available in Amazon S3.

PART 1 – CREATE AMAZON S3 BUCKET

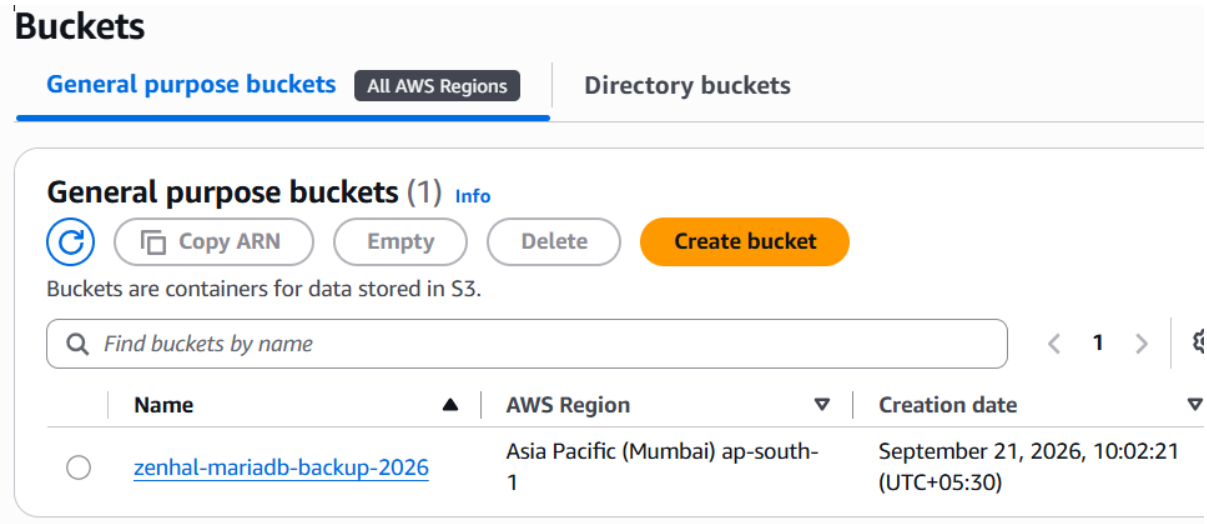
Step 1 – Create an S3 Bucket

An Amazon S3 bucket was created to store the MariaDB database backup files.

Bucket name:

```
zenhal-mariadb-backup-2026
```

The S3 bucket acts as the remote storage location for the database backups.



PART 2 – CREATE IAM POLICY

Step 2 – Create IAM Policy

An IAM policy was created to provide the EC2 instance with the minimum required permissions to access the S3 bucket.

Policy name:

`MariaDBBackupToS3Policy`

The policy allows the following actions:

```
s3:ListBucket
s3:PutObject
```

The permissions were restricted to the specific S3 bucket:

`zenhal-mariadb-backup-2026`

Policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket"
      ],
      "Resource": "arn:aws:s3:::zenhal-mariadb-backup-2026"
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::zenhal-mariadb-backup-2026/*"
    }
  ]
}
```

PART 3 – CREATE AND ATTACH IAM ROLE

Step 3 – Create IAM Role

An IAM Role was created for the EC2 instance.

Role name:

`MariaDBBackupEC2Role`

The previously created `MariaDBBackupToS3Policy` was attached to this role.

Step 4 – Attach the Role to EC2

The `MariaDBBackupEC2Role` was attached to the RHEL EC2 instance.

The IAM Role allows the EC2 instance to access S3 without storing permanent AWS access keys and secret keys on the server.

PART 4 – CONFIGURE AWS CLI

Step 5 – Install AWS CLI

AWS CLI was configured on the EC2 server to communicate with AWS services from the Linux terminal.

The IAM Role attached to the EC2 instance provides the required temporary credentials automatically.

Step 6 – Verify IAM Role

The following command was used:

```
aws sts get-caller-identity
```

The output confirmed that the EC2 instance was using:

```
MariaDBBackupEC2Role
```

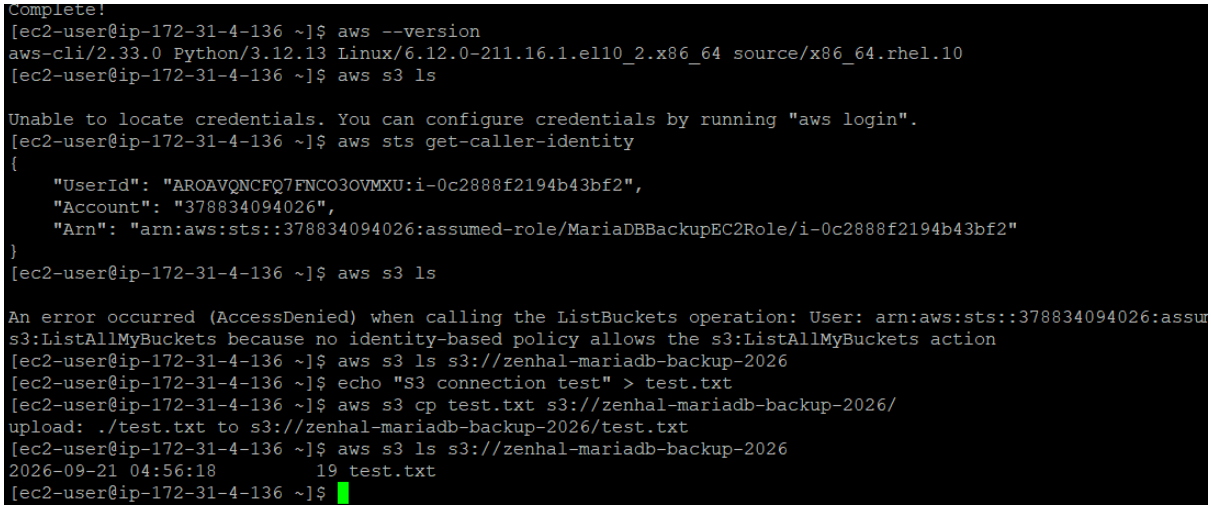
This verified that the IAM Role was successfully attached and available to the EC2 instance.

Step 7 – Verify S3 Bucket Access

The following command was used:

```
aws s3 ls s3://zenhal-mariadb-backup-2026/
```

The bucket was successfully accessed through AWS CLI.



PART 5 – CONFIGURE MARIADB SERVER

Step 8 – Install MariaDB

Since the EC2 instance was running RHEL, the MariaDB server package was installed using:

```
sudo dnf install -y mariadb-server
```

Step 9 – Start and Enable MariaDB

MariaDB was started and configured to start automatically during system boot:

```
sudo systemctl enable --now mariadb
```

Step 10 – Access MariaDB

The MariaDB console was accessed using:

```
sudo mysql
```

PART 6 – CREATE DATABASE AND SAMPLE DATA

Step 11 – Create Database

A database named `backup_test` was created:

```
CREATE DATABASE backup_test;
```

The database was selected:

```
USE backup_test;
```

Step 13 – Insert Sample Data

Sample records were inserted:

```
INSERT INTO students (name, course)
VALUES
('Zenhal', 'Cloud and DevOps'),
('Student2', 'Linux');
```

The data was verified using:

```
SELECT * FROM students;
```

The database was now ready to be backed up.

PART 7 – CREATE BACKUP DIRECTORY

Step 14 – Create Backup Directory

A separate directory was created to store the database backup files:

```
sudo mkdir -p /backup
```

The `/backup` directory was used as the source directory for the S3 synchronization.

PART 8 – CREATE MARIADB BACKUP

Step 15 – Take Manual Database Backup

The `mysqldump` utility was used to create a logical SQL backup of the `backup_test` database.

The backup was created using:

```
sudo mysqldump backup_test | sudo tee /backup/backup_test.sql
> /dev/null
```

The resulting file was:

```
/backup/backup_test.sql
```

`mysqldump` stores the database structure and data as SQL statements such as `CREATE TABLE` and `INSERT INTO`.

Step 16 – Verify Backup

The backup directory was checked:

```
ls -lh /backup/
```

The backup file was successfully created.

The contents were also checked using:

```
sudo grep -E "CREATE TABLE|INSERT INTO"
/backup/backup_test.sql
```

The output confirmed the presence of the table structure and data.

```
[ec2-user@ip-172-31-4-136 ~]$ sudo mkdir -p /backup
[ec2-user@ip-172-31-4-136 ~]$ ls -l /backup
total 0
[ec2-user@ip-172-31-4-136 ~]$ ls -ld /backup
drwxr-xr-x. 2 root root 6 Sep 21 05:27 /backup
[ec2-user@ip-172-31-4-136 ~]$  sudo mysqldump backup_test > /backup/backup_test.sql
-bash: /backup/backup_test.sql: Permission denied
[ec2-user@ip-172-31-4-136 ~]$ sudo mysqldump backup_test | sudo tee /backup/backup_test.sql > /dev/null
[ec2-user@ip-172-31-4-136 ~]$ ls -lh /backup/
total 4.0K
-rw-r--r--. 1 root root 2.1K Sep 21 05:34 backup_test.sql
[ec2-user@ip-172-31-4-136 ~]$ sudo grep -E "CREATE TABLE|INSERT INTO" /backup/backup_test.sql
CREATE TABLE `students` (
INSERT INTO `students` VALUES
```

PART 9 – CREATE BACKUP SCRIPT

Step 17 – Create Shell Script

A shell script was created:

```
sudo vi /usr/local/bin/mariadb_backup.sh
```

The following script was added:

```
#!/bin/bash

BACKUP_DIR="/backup"
DATE=$(date +%Y-%m-%d)

sudo mysqldump backup_test | sudo tee
"$BACKUP_DIR/backup_test-$DATE.sql" > /dev/null
```

The script automatically creates a date-based backup filename.

For example:

```
backup_test-2026-09-21.sql
```

Step 18 – Give Execute Permission

Execute permission was given to the script:

```
sudo chmod +x /usr/local/bin/mariadb_backup.sh
```

Step 19 – Run the Backup Script

The script was executed:

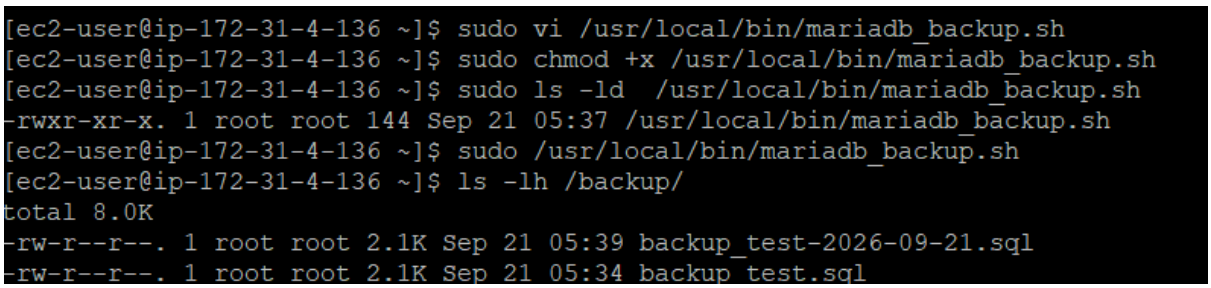
```
sudo /usr/local/bin/mariadb_backup.sh
```

The backup directory was checked:

```
ls -lh /backup/
```

The following backup files were successfully created:

```
backup_test.sql
backup_test-2026-09-21.sql
```



PART 10 – SYNCHRONIZE BACKUPS WITH S3

Step 20 – Sync Backup Directory with S3

The `/backup` directory was synchronized with the S3 bucket using:

```
aws s3 sync /backup s3://zenhal-mariadb-backup-2026/
```

The command successfully uploaded the backup files:

```
backup_test.sql
backup_test-2026-09-21.sql
```

The synchronization process is:

```
MariaDB
   ↓
mysqldump
   ↓
/backup/
   ↓
AWS CLI
   ↓
IAM Role
   ↓
Amazon S3
```

PART 11 – VERIFY BACKUPS IN S3

Step 21 – Verify S3 Objects

The contents of the S3 bucket were checked using:

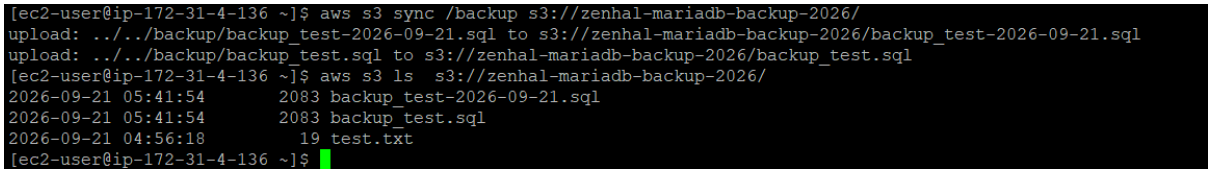
```
aws s3 ls s3://zenhal-mariadb-backup-2026/
```

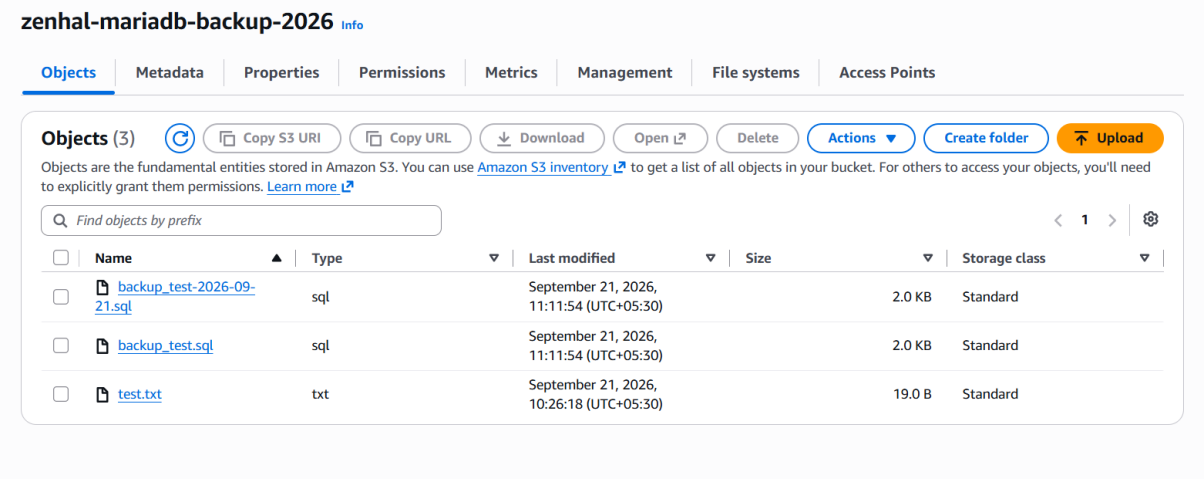
The output showed:

```
backup_test-2026-09-21.sql
backup_test.sql
```

This confirmed that the MariaDB backup files were successfully uploaded to Amazon S3.

The same backup files were also verified through the Amazon S3 Management Console.





VERIFICATION

The following points were successfully verified:

1. The S3 bucket `zenhal-mariadb-backup-2026` was created.
2. The `MariaDBBackupToS3Policy` IAM policy was created.
3. The `MariaDBBackupEC2Role` IAM Role was attached to the EC2 instance.
4. AWS CLI successfully used the IAM Role to access S3.
5. MariaDB was successfully installed and configured.
6. The `backup_test` database was created with sample data.
7. `mysqldump` successfully created the SQL backup.
8. A shell script successfully generated a date-based backup.
9. The `/backup` directory was successfully synchronized with S3.
10. The backup files were visible in the S3 bucket.

CONCLUSION

The MariaDB database backup process was successfully implemented on the RHEL EC2 instance. A shell script was created to generate SQL backups using `mysqldump`, and AWS CLI was used to synchronize the backup directory with the Amazon S3 bucket.

An IAM Role was used to provide secure S3 access to the EC2 instance without storing permanent AWS access keys on the server. The successful appearance of the `.sql` backup files in the S3 bucket confirmed that the complete backup-to-S3 workflow was working successfully.

Note: The practical demonstrated backup of the `backup_test` database. It did not configure an automated schedule such as cron, and it did not configure backups of every MariaDB database.